

UP LT GRADE • COMPUTER SCIENCE MAINS

Computer Organization & Architecture — Sample Paper with Model Answers

Following the Official Mains Exam Pattern

Total Questions: 20 (Section A: 10, Section B: 10)

Section A: 8 marks each, answer in ~125 words — 80 marks total

Section B: 12 marks each, answer in ~200 words — 120 marks total

Model answers are written strictly within word limits, exam-style, with scoring tips.

Covers all 16 COA topics • 45-day study plan practice set

How to Use This Sample Paper

Purpose: This paper mirrors the real UPPSC LT Grade Mains pattern (as seen in the conducted Science paper) and covers every COA topic in your syllabus. Each question includes a model answer written to the EXACT word budget you'll face in the real exam.

Why Word Limits Matter

Analysis of the actual conducted Science paper showed that most high-mark questions are **compound** — they ask 2-4 things in one question. The word limit, not lack of knowledge, is usually what costs marks. A student who deeply understands a topic but spends all 200 words on only one part of a 4-part question loses more than half the marks. Every model answer below deliberately addresses **every sub-part** of the question within the word budget.

How to Practice With This Paper

1. Attempt each question yourself first, with a timer (roughly 9 minutes for Section A, 13 minutes for Section B, to fit the 3-hour/20-question format).
2. Compare your answer's structure against the model answer — not just content, but how many distinct points it hits.
3. Read the "Exam Strategy Tip" under each answer — these explain WHY the answer is structured that way, so you can replicate the technique on unseen questions.

Topics Covered

Stored Program Concept · Components of Computer System · Opcode/Operand · Instruction Cycle · CPU Organization & ALU · Control Unit Types · GPR/SPR Registers · Memory Organization · I/O Organization & DMA · Functioning of CPU · Instruction Formats · Instruction Types · Addressing Modes · RAM/ROM · Multi-core Architecture · Multiprocessor/Multicomputer

Explain the Stored Program Concept. Why is it considered fundamental to modern computer architecture?

8 Marks

Topic: Stored Program Concept | Target length: ~125 words

Model Answer 121 words

The Stored Program Concept, proposed by John von Neumann in 1945, states that both program instructions and data are stored together in the same memory in binary form, and the CPU fetches, decodes, and executes these instructions sequentially using a Program Counter (PC). Earlier computers were hard-wired, requiring physical rewiring of plugs and switches for every new task — slow and inflexible. Storing programs in memory allows a computer's function to be changed simply by loading a new program, without touching the hardware. This concept underlies the Fetch-Decode-Execute cycle followed by every modern CPU. Since a single bus carries both instructions and data between CPU and memory, this design also causes the Von Neumann Bottleneck, later addressed by the Harvard Architecture.

Exam Strategy Tip

Mention: (1) von Neumann's name & year, (2) before/after contrast, (3) link to Fetch-Decode-Execute, (4) one limitation (bottleneck) — four sub-points in one crisp answer.

Draw and explain the block diagram of the basic components of a computer system.

8 Marks

Topic: Components of a Computer System | Target length: ~125 words

Model Answer 109 words

A computer system is built from five core components. The Input Unit (keyboard, mouse, scanner) accepts data and converts it to machine-readable form. The Central Processing Unit (CPU) — containing the ALU, Control Unit, and Registers — is the 'brain' that fetches and executes instructions. The Memory Unit stores data and programs, split into Primary memory (RAM/ROM) and Secondary memory (HDD/SSD). The Output Unit (monitor, printer) converts processed results into human-readable form. Finally, the Bus/Interconnection system (Address, Data, and Control buses) carries signals between all units.

Block Diagram: Input Unit → CPU (+Memory) → Output Unit, all linked via the system bus, following the classic IPOS (Input-Process-Output-Storage) data flow.

Exam Strategy Tip

Always draw the block diagram (Input → CPU+Memory → Output with bus lines) — diagram marks are separate from explanation marks.

Differentiate between Opcode and Operand with the help of a suitable machine instruction example.

8 Marks

Topic: Opcode and Operand | Target length: ~125 words

Model Answer 125 words

A machine instruction is a binary-encoded command made of two main parts. The Opcode (Operation Code) specifies WHAT operation the CPU must perform — e.g., ADD, SUB, MOV, LOAD. The Operand specifies WHICH data the operation works on — this can be a register, a memory address, or an immediate constant.

Example: In the instruction ADD R1, R2 — 'ADD' is the Opcode (the operation), while 'R1' and 'R2' are the Operands (the registers holding the data). The Control Unit decodes the opcode to know which operation to trigger, while the ALU uses the operand values as its actual input data. Operands may be Register, Memory, or Immediate type, each with different access speed — register operands being fastest since no memory access is needed.

Exam Strategy Tip

Always label a real instruction example (ADD R1,R2) explicitly identifying opcode vs operand — examiners specifically reward this.

Explain the Fetch-Decode-Execute cycle, describing the role of PC, IR, MAR, and MDR at each stage.

8 Marks

Topic: Instruction Cycle | Target length: ~125 words

Model Answer

120 words

The Instruction Cycle is the repeated sequence every CPU follows to run a program. In the Fetch stage, the Memory Address Register (MAR) is loaded from the Program Counter (PC); the instruction at that address is copied into the Memory Data Register (MDR) and then into the Instruction Register (IR), while PC increments to point to the next instruction. In the Decode stage, the Control Unit interprets the opcode in the IR to identify the operation and operand locations. In the Execute stage, the ALU performs the operation and results are written back to a register or memory. Finally, the CPU checks for pending interrupts before repeating the cycle. This loop, repeating billions of times per second, is program execution.

Exam Strategy Tip

Write the Register Transfer Notation ($MAR \leftarrow PC$; $MDR \leftarrow M[MAR]$; $IR \leftarrow MDR$) if space allows — RTN steps are heavily rewarded.

Differentiate between General Purpose Registers and Special Purpose Registers, giving two examples of each.

8 Marks

Topic: GPR vs SPR | Target length: ~125 words

Model Answer 122 words

Registers are the CPU's fastest storage locations, but they serve two distinct roles. General Purpose Registers (GPRs) are multi-purpose registers (R0, R1, R2...) that the programmer or compiler can use flexibly to hold any data, address, or intermediate result — e.g., R0 and R1 in an ARM processor. Special Purpose Registers (SPRs), in contrast, are reserved for one specific, fixed CPU function and cannot be repurposed — e.g., the Program Counter (PC), which always holds the address of the next instruction, and the Instruction Register (IR), which holds the instruction currently being decoded. Other SPRs include MAR, MDR, the Accumulator, and the Status/Flag register. In short: GPRs give the programmer flexibility, while SPRs provide the essential plumbing for the Fetch-Decode-Execute cycle itself.

Exam Strategy Tip

List at least 2 named SPRs with their exact function (not just 'PC' but what it does) — this precision earns full marks.

Differentiate between Hardwired and Microprogrammed Control Units.

8 Marks

Topic: Control Unit types | Target length: ~125 words

Model Answer 123 words

The Control Unit generates the timing and control signals needed to execute instructions, and can be designed in two ways. A Hardwired Control Unit uses fixed logic circuits — gates, flip-flops, and decoders — to directly generate control signals from the instruction opcode. It is very fast but difficult to modify once built, making it ideal for RISC processors with simple, fixed instruction sets. A Microprogrammed Control Unit instead stores 'microinstructions' in a Control Memory (ROM); each opcode maps to a starting address, and the sequencer reads out microinstructions whose control words activate the needed signals. It is slower due to the extra memory access but far easier to modify or debug, making it suitable for CISC processors with large, complex instruction sets.

Exam Strategy Tip

A quick two-column comparison (Speed vs Flexibility) at the end strengthens a 'differentiate' answer.

Explain any three addressing modes with suitable examples.

8 Marks

*Topic: Addressing Modes | Target length: ~125 words***Model Answer** 122 words

An addressing mode defines how the CPU determines the effective address of an instruction's operand. In Immediate Addressing, the operand VALUE is given directly in the instruction, e.g., `MOV R1, #5` — fastest, since no memory access is needed. In Direct Addressing, the instruction contains the actual memory address of the operand, e.g., `MOV R1, [2000]` — one memory access required. In Indirect Addressing, the instruction's address field points to a memory location that itself CONTAINS the operand's real address (a pointer) — this needs two memory accesses but supports a much larger effective address range, making it essential for implementing pointers and linked lists. Register and Indexed (Base+Offset) addressing are two other common modes, the latter being ideal for accessing arrays.

Exam Strategy Tip

Pick modes with clearly different mechanisms (Immediate/Direct/Indirect) so the contrast is obvious, and give one example instruction per mode.

Differentiate between RAM and ROM. What are the different types of ROM?

8 Marks

*Topic: RAM vs ROM | Target length: ~125 words***Model Answer**

111 words

RAM (Random Access Memory) is volatile — it loses all data when power is switched off — and supports both read and write access; it holds the running programs and OS, and comes in two forms: SRAM (uses flip-flops, very fast, used as CPU cache) and DRAM (uses capacitors, needs periodic refreshing, cheaper, used as main memory). ROM (Read-Only Memory) is non-volatile — it retains data without power — and is mainly read-only, storing firmware, BIOS, and the bootstrap loader. ROM itself has several types: PROM (programmable once by the user), EPROM (erasable using ultraviolet light and then reprogrammable), and EEPROM (erasable electrically, byte-by-byte — the basis of modern Flash memory).

Exam Strategy Tip

A one-line table (Volatile? Read/Write? Use-case?) for RAM vs ROM is the fastest way to hit full marks within 125 words.

Explain Direct Memory Access (DMA) and how it differs from Interrupt-driven I/O.

8 Marks

Topic: DMA vs Interrupt I/O | Target length: ~125 words

Model Answer 123 words

Direct Memory Access (DMA) is a data-transfer technique where a dedicated DMA controller moves data directly between an I/O device and memory, without CPU involvement in each word transferred. The CPU only initiates the transfer (giving source, destination, and count) and is interrupted just once, when the entire transfer completes — making DMA ideal for large, fast transfers like disk-to-memory. In Interrupt-driven I/O, by contrast, the CPU issues a request and continues other work, but the device interrupts the CPU each time it is ready, and the CPU itself must transfer each individual data word. This makes interrupt-driven I/O suitable for slow, small transfers (like keyboard input) but far less efficient than DMA for bulk data movement, since the CPU stays involved throughout.

Exam Strategy Tip

State clearly WHO moves the data in each case (DMA controller vs CPU itself) — that single distinction is the crux of this answer.

What is Multi-core architecture? How does it differ from simply increasing the clock speed of a single core?

8 Marks

Topic: Multi-core Architecture | Target length: ~125 words

Model Answer 119 words

A multi-core processor integrates two or more independent processing cores onto a single chip, each with its own ALU, registers, and often private cache, sharing memory and other resources through an on-chip interconnect to execute multiple instruction streams in parallel. This differs fundamentally from the older approach of increasing a single core's clock speed, which hit practical limits around 3–4 GHz due to excessive heat and power consumption, with diminishing performance returns. Instead of one super-fast core, multi-core design places several moderate-speed cores on one chip, giving much better performance-per-watt — but only for multi-threaded software, since a single-threaded program cannot automatically benefit from extra cores. Examples include dual-core, quad-core, and octa-core CPUs used in modern laptops and phones.

Exam Strategy Tip

Explicitly contrast 'higher GHz' (old approach) vs 'more cores' (modern approach) — this is the exact comparison examiners test.

Explain the organization of the CPU with focus on the ALU and Control Unit. How do they coordinate to execute an instruction? Draw a block diagram.

12 Marks

Topic: CPU Organization & ALU | Target length: ~200 words

Model Answer 181 words

The CPU is organized into three main parts: the Arithmetic Logic Unit (ALU), which performs all arithmetic (add, subtract) and logical (AND, OR, NOT, compare) operations; the Control Unit (CU), which generates the timing and control signals that direct every other component, deciding WHEN and WHAT happens; and Registers, which provide fast temporary storage (Accumulator, PC, IR, MAR, MDR, and general-purpose registers). All three are connected by an internal data bus.

Coordination during execution: when an instruction like ADD R1, R2 is fetched, the Control Unit decodes the opcode and issues the ALU's operation-select signal, telling it to perform addition rather than any other operation. The ALU then reads operand values from the registers into its input latches, performs the computation using combinational logic, and writes the result back to a register — while the Control Unit also updates the Status/Flag register (Zero, Carry, Sign, Overflow) based on the outcome.

Block Diagram: Control Unit → (control signals) → ALU → (data) → Registers, all interconnected, with the CU acting as the 'manager' and the ALU as the 'calculator' of the CPU.

Exam Strategy Tip

Structure: (1) name all 3 parts, (2) explain ALU role, (3) explain CU role, (4) walk through ONE example instruction showing coordination, (5) diagram. Five distinct parts = five distinct scoring points.

What are the different instruction formats (zero, one, two, and three-address)? Explain each with an example and discuss the trade-off between instruction length and number of instructions needed.

Topic: Instruction Formats | Target length: ~200 words

Model Answer 175 words

An instruction format defines how opcode and operand fields are arranged within an instruction, determining how many addresses (0 to 3) it can directly reference.

Zero-Address Format: No explicit operand fields; uses a stack, with operations implied via PUSH/POP (e.g., stack-based calculators).

One-Address Format: Uses an implied Accumulator as the second operand — e.g., ADD X means $ACC = ACC + X$.

Two-Address Format: One operand doubles as both source and destination — e.g., ADD R1, R2 means $R1 = R1 + R2$, overwriting R1.

Three-Address Format: Separate fields for two sources and one destination — e.g., ADD R1, R2, R3 means $R1 = R2 + R3$, preserving both original operands.

Trade-off: Fewer address fields (0/1-address) give shorter, denser instructions but require MORE instructions (and extra MOV operations) to complete a computation without losing data. More address fields (3-address) give longer instructions but need FEWER total instructions, since results don't overwrite sources — common in RISC machines like ARM and MIPS, simplifying compiler code generation, while 2-address formats are typical of CISC machines like x86.

Exam Strategy Tip

Give one worked example per format type (4 examples total) then explicitly state the length-vs-count trade-off as a separate closing point — this hits both 'explain' and 'discuss' halves of the question.

Explain the different types of instructions (Data Transfer, Arithmetic, Logical, Control Transfer, I/O) with one example mnemonic each. Also explain how conditional branching works using status flags.

12 Marks

Topic: Instruction Types | Target length: ~200 words

Model Answer

190 words

CPU instructions fall into five broad categories. Data Transfer instructions move data between registers, memory, and I/O — e.g., MOV R1, R2. Arithmetic instructions perform mathematical operations — e.g., ADD R1, R2. Logical instructions perform bit-level operations — e.g., AND R1, R2, used for masking and bit manipulation. Control Transfer instructions change the normal sequential execution order — e.g., JMP LABEL (unconditional) or CALL/RET for subroutines. Input/Output instructions transfer data between the CPU and external devices — e.g., IN PORT1.

Conditional Branching using Status Flags: Instructions like CMP or SUB automatically update Status Flags — Zero (Z), Carry (C), Sign (S), Overflow (O) — based on their result. A conditional jump instruction, such as JZ (Jump if Zero), then checks the relevant flag: if CMP found the two values equal, the Zero flag is set to 1, and JZ transfers control to the target label; otherwise, execution simply continues to the next instruction. This flag-checking mechanism is exactly how if-else logic and loops are implemented at the hardware level — without conditional branches based on flags, every program would execute the exact same sequence every single time, regardless of input.

Exam Strategy Tip

Give exactly one real mnemonic per category (5 total), then dedicate a full paragraph to the flags mechanism since that's explicitly asked separately — don't let it become an afterthought.

Describe the complete memory hierarchy from Registers to Secondary memory. Explain why this hierarchy exists and how it balances speed, cost, and capacity.

12 Marks

Topic: Memory Hierarchy | Target length: ~200 words

Model Answer 200 words

The memory hierarchy arranges storage into layers, from fastest/smallest at the top to slowest/largest at the bottom. Registers sit inside the CPU itself — fastest possible access, but only a few bytes of capacity. Cache Memory (L1/L2/L3) sits between the CPU and RAM, built from fast SRAM, storing frequently used data to reduce average access time — capacity in the KB to MB range. Primary Memory (RAM) is the main working memory — volatile, holding running programs, with capacity in GBs. Secondary Memory (HDD/SSD) is non-volatile and holds long-term data — slowest of the four, but with capacity in the TB range.

Why this hierarchy exists: fast memory (registers, cache) is physically expensive to build at large sizes, while large memory (disk) is cheap but inherently slow due to its underlying technology. No single memory type can be simultaneously fast, large, AND cheap. By layering multiple types together, the system exploits the Principle of Locality — programs tend to reuse the same or nearby data repeatedly — so the fast, small, expensive layers serve most requests, while the slow, large, cheap layers store the bulk of data rarely accessed directly. This design gives near-cache speed at near-disk cost and capacity.

Exam Strategy Tip

Draw the 4-level pyramid diagram (Registers→Cache→RAM→Disk) and explicitly state the speed ↓ / capacity ↑ / cost ↓ trend as you go down — this diagram alone often carries 3-4 of the 12 marks.

Explain Direct, Indirect, and Indexed addressing modes with examples. For a Base register R containing 3000 and an Index/Offset of 25, calculate the effective address under Indexed addressing.

12 Marks

Topic: Addressing Modes (calculation) | Target length: ~200 words

Model Answer 211 words

In Direct Addressing, the instruction's address field IS the effective address of the operand — e.g., `MOV R1, [2000]` fetches data directly from memory location 2000, needing only one memory access.

In Indirect Addressing, the address field instead points to a memory location that CONTAINS the real address of the operand (a pointer). For example, if the instruction references location A, and location A holds the value B, the CPU must first read A to get B, then read B to get the actual operand — two memory accesses in total, but this supports a much larger effective address range, and is essential for implementing pointers and dynamic data structures.

In Indexed Addressing, the Effective Address is computed as Base Register value + Index/Offset given in the instruction — ideal for accessing elements of an array, where the base points to the array's start and the index selects a specific element.

Calculation: Given Base Register R = 3000 and Offset = 25:

Effective Address = Base + Offset = $3000 + 25 = 3025$.

So the CPU would access memory location 3025 to retrieve the operand — this pattern lets a single instruction access different array elements just by changing the offset value, without needing a new instruction for each element.

Exam Strategy Tip

Numerical addressing-mode questions are common in Section B — always show the formula ($EA = \text{Base} + \text{Offset}$) explicitly before substituting numbers, then state the final address clearly.

Compare Multiprocessor and Multicomputer systems in terms of memory architecture, coupling, and OS control. Give one real-world example of each.

12 Marks

Topic: Multiprocessor vs Multicomputer | Target length: ~200 words

Model Answer 188 words

A Multiprocessor system tightly couples two or more CPUs around a COMMON shared memory, all controlled by a single operating system. Since all processors can directly read/write the same physical memory, communication between them is very fast, but requires careful synchronization to avoid conflicts when multiple CPUs access the same memory location simultaneously. A real-world example is a multi-socket server, where several CPU chips share one large memory pool under one OS instance.

A Multicomputer system, in contrast, loosely couples multiple independent computers, each with its OWN private memory and its own copy of the operating system, connected via a network. Since there is no shared memory, nodes communicate only through explicit message passing over the network — higher latency than shared memory, but this design scales far more easily to thousands of nodes. A real-world example is a computer cluster or a cloud data center, where independent machines coordinate via network messages to jointly process massive workloads.

Summary: Multiprocessor = one OS, shared memory, tightly coupled, fast but harder to scale; Multicomputer = many OS copies, private memory, loosely coupled, slower per-message but scales to enormous size.

Exam Strategy Tip

Structure your answer around the three asked dimensions in order (memory → coupling → OS control) so the examiner can tick off each one — then close with one example per system as explicitly requested.

Explain the working of a Full Adder. Derive its Sum and Carry expressions and show how it is used to build a Ripple Carry Adder.

12 Marks

Topic: Full Adder / Ripple Carry Adder | Target length: ~200 words

Model Answer 209 words

A Full Adder is a combinational circuit that adds three single bits: two operand bits (A, B) and a Carry-in (Cin) from a previous stage, producing a Sum bit and a Carry-out bit. Unlike a Half Adder (which only adds two bits with no carry-in), a Full Adder can be chained together with others to add multi-bit numbers.

From its truth table, the Sum output is 1 whenever an odd number of inputs are 1, giving:

Sum = $A \oplus B \oplus C_{in}$ (a 3-input XOR).

The Carry-out is 1 whenever at least two of the three inputs are 1, giving:

Cout = $AB + BC_{in} + AC_{in}$.

Building a Ripple Carry Adder: To add two n-bit binary numbers, n Full Adders are chained together, where each Full Adder's Carry-out output feeds directly into the next (more significant) Full Adder's Carry-in input — like a ripple flowing from the least significant bit to the most significant bit. The first (LSB) stage typically uses a Half Adder or a Full Adder with $C_{in}=0$. While simple to build, a Ripple Carry Adder's total addition time grows with the number of bits, since each stage must wait for the carry from the previous stage — this delay is called carry propagation delay.

Exam Strategy Tip

Always include the truth table derivation logic behind Sum/Carry formulas (not just state them), then explicitly describe the 'chaining' mechanism — this is a classic full-diagram-plus-derivation question.

Explain Programmed I/O, Interrupt-driven I/O, and DMA. Compare their efficiency and CPU involvement with a suitable example.

12 Marks

Topic: I/O Transfer Techniques | Target length: ~200 words

Model Answer 216 words

Programmed I/O is the simplest technique: the CPU continuously polls (repeatedly checks) the device's status register in a loop until it is ready, then transfers the data itself. This wastes significant CPU time, since the CPU can do nothing else while waiting — suitable only for very simple, low-speed systems.

Interrupt-driven I/O improves on this: the CPU issues a request to the device and continues executing other instructions; the device then sends an interrupt signal to the CPU only when it is ready. The CPU pauses its current task, transfers the data itself, then resumes — much more efficient than polling for slow, small transfers like keyboard or mouse input, since the CPU is free during the wait.

Direct Memory Access (DMA) is the most efficient technique for large transfers: a separate DMA controller is given the source, destination, and byte count by the CPU, and then handles the ENTIRE data transfer directly between the device and memory without further CPU involvement, interrupting the CPU only once, when the whole transfer is complete.

Comparison: Programmed I/O has the highest CPU involvement (worst efficiency), Interrupt-driven I/O reduces CPU involvement to only the actual transfer moments, and DMA has the lowest CPU involvement (best efficiency) — ideal for bulk transfers like loading a file from disk into memory.

Exam Strategy Tip

A one-line summary sentence ranking all three by CPU involvement (as the last line here) is a fast way to explicitly answer the 'compare' part without needing a separate table.

Explain the functioning of the CPU highlighting the role of the system clock, control unit, and buses. How does Single-cycle differ from Multi-cycle CPU design?

12 Marks

Topic: Functioning of CPU | Target length: ~200 words

Model Answer

209 words

The CPU functions by continuously executing the Fetch-Decode-Execute cycle, with every internal action precisely synchronized. The System Clock generates regular timing pulses that coordinate every micro-operation across the CPU — its frequency (measured in GHz) determines how fast this cycle repeats. The Control Unit acts as the 'conductor,' decoding each instruction's opcode and issuing the exact control signals, at the exact clock pulse, needed to activate the ALU, registers, or memory as required. Buses — Address, Data, and Control — serve as the communication highways, carrying addresses, data values, and control signals between the CPU, memory, and I/O devices throughout execution.

Single-cycle CPU design: Every instruction, regardless of complexity, is forced to complete in exactly one clock cycle. This gives simple control logic, but the clock speed must be slow enough to accommodate even the SLOWEST, most complex instruction — wasting time on simple instructions that could finish much faster.

Multi-cycle CPU design: Instructions take a variable number of clock cycles depending on their actual complexity — a simple instruction might take 2 cycles, a complex one 5 — allowing the clock itself to run much faster overall. This is more efficient on average, and is the foundation that modern pipelined CPUs build further upon for even greater throughput.

Exam Strategy Tip

Cover clock, CU, and buses as three distinct labeled points first, then use a clear 'Single-cycle vs Multi-cycle' sub-heading for the second half — two-part questions score better when visibly split like this.

A memory chip has 1024×8 capacity. (a) How many address lines and data lines are needed? (b) How many such chips are needed to build a $16K \times 8$ memory system? Also (c) explain Memory-Mapped I/O vs Isolated I/O.

12 Marks

Topic: Memory Sizing + I/O Addressing | Target length: ~200 words

Model Answer 191 words

(a) Address and Data Lines: $1024 = 2^{10}$ memory locations, so 10 address lines are needed to uniquely address every location. Since each location stores 8 bits, 8 data lines are needed to carry that data in/out. Total: 10 address lines, 8 data lines.

(b) Number of Chips: $16K = 16384 = 2^{14}$ locations required in total. Each chip provides $1024 = 2^{10}$ locations. Number of chips = $16384 \div 1024 = 16$ chips. These 16 chips are arranged side-by-side, using extra address-decoding lines to select which chip is active, extending the address range while keeping the 8-bit data width unchanged (since each chip already matches the required 8-bit width).

(c) Memory-Mapped I/O vs Isolated I/O: In Memory-Mapped I/O, I/O devices share the SAME address space as memory — the CPU uses ordinary memory-read/write instructions (like MOV) to access devices, simplifying instruction design but reducing the address space available for actual memory. In Isolated (Port-Mapped) I/O, devices have a SEPARATE address space accessed only via special IN/OUT instructions, keeping the full memory address space free for memory alone, at the cost of needing dedicated I/O instructions in the CPU's instruction set.

Exam Strategy Tip

For sizing questions, always show the power-of-2 conversion explicitly ($1024=2^{10}$, $16K=2^{14}$) before dividing — examiners give method marks even if the final chip count has a small error.

— End of Sample Paper —

Computer Organization & Architecture | UP LT Grade CS Mains Preparation